[Experiment, Analysis, and Benchmark] Decide on Arrival: Placement Timing Beats Placement Prediction on Production KV Traces

Suan Lee
Semyung University
Jecheon, Republic of Korea
suanlee@semyung.ac.kr
ORCID 0000-0002-3047-1167

***Abstract*—A natural way to evaluate how a memory system for LLM agents should place KV-cache blocks across storage tiers, and the one we used for three revisions of this work, is at a split instant: observe a prefix, place the blocks that exist, and bill the result against a hindsight oracle. On two production Mooncake traces this protocol measures almost nothing. Of the oracle's advantage, 94.7% and 95.9% sits on blocks that do not yet exist at the split, so the best possible split-time policy reaches 5.3% and 4.1% of it; the share stays at 92–99% for every split from 0.3 to 0.7 and under tenfold changes to the latency and token weights. Inside that protocol a better ranker helps monotonically, but only over the sliver: the strongest model gains under 3% of the oracle's advantage. Deciding at each block's arrival instead puts most of the advantage within reach: every arrival-time rule we test captures 45–85% of it, and there the reuse a rule assumes for a block it has never seen moves the result by 30–39 points, while how it ranks those blocks moves it by about one. On Mooncake's synthetic control the margin over the split-time ceiling narrows and is not robust. Reactive tiering is consistent with this: LRU, LFU and a fill-once cache reach 75–99% and 40–94% of Belady's hit rate, the best adaptive policy (ARC, LeCaR, CACHEUS) beats the best of those rules, each chosen per operating point, by at most 4.6 points, and a learned replacement policy (LRB) stays below ARC, because 76–78% of blocks are never reused. We give a one-line diagnostic for the protocol failure, release an artifact that regenerates every number on CPU, and catalogue the measurement defects we made and caught, including leaks in our own arrival-time experiment that survived every number check.**

***Index Terms*—KV cache, memory tiering, cache replacement, evaluation methodology, LLM agents, reproducibility**

I. Introduction

Large language model (LLM) agents increasingly depend on memory that outlives a single context window—the facts they extract, the turns they have seen, the tools they have run, the summaries they have written. How well an agent answers, how fast it responds, and how much it costs are all decided by one question asked of every memory item: **in what form should it be kept, where should it live, and when should it be (re)materialized?** Concretely, each item can be stored as plaintext, a vector, a graph, a latent/KV activation, or baked into parameters; it can sit on GPU, CPU, disk, or a remote tier; and it can be written inline while serving, consolidated during an idle window, or left untouched.

Today this single decision is **fragmented across three research communities**, each of which optimizes one axis and freezes the others. Memory-operating-system work—MemOS [1], MemGPT [2]—provides the primitives to migrate an item between representations and tiers, but *decides when to migrate by hand-tuned heuristics*. Reinforcement-learned memory managers—Memory-R1 [3], Memory-as-Action [4]—learn *what content* to add, update, or delete, but operate over a single fixed representation and never choose a tier or schedule an offline write. Systems-level cache tiering—LMCache [5]—places bytes across GPU/CPU/disk/remote to minimize *miss rate or time-to-first-token*, with no notion of the downstream task utility that the memory actually serves.

Our central observation is that **these three axes are coupled and cannot be optimized separately.** Promoting an item to a latent or parameter representation is an expensive offline write, but it is then cheap and fast to serve; leaving it as plaintext on disk is cheap to write but slow to read and heavy on context tokens. The best representation depends on the tier and on how often the item will be reused; the best tier depends on the representation's serving cost and the item's size; and whether to pay the promotion cost at all depends on the budget. This is precisely a **budget-constrained scheduling problem**—and a large body of ML-for-systems work (Belady-oracle imitation for cache replacement [6], GNN schedulers for clusters [7], offline-RL cache eviction in production [8]) demonstrates that such problems are learnable.

The reason to act now is that every substrate exists but nothing orchestrates them: MemCube migration, RL memory managers, tiered KV caches, latent memory [9], and sleep-time consolidation [10] all landed by late 2025, unconnected. What is missing is a *learned scheduler* that, per item and under a shared budget, trades accuracy against latency and cost, with **task utility—not miss rate—as the objective.**

We present **STOA** (Storage & Tiered Orchestration for Agents), a learned control plane that decides **representation $\times$ tier $\times$ timing jointly**, per memory item, as a single budget-constrained Markov decision process. The operator sets a latency/cost/token budget (L, C, B) at serving time and one policy adapts, tracing the accuracy–latency–cost frontier

without retraining. To tame the combinatorics (fifty legal actions per item, across many items) STOA factors the policy into representation, tier, and timing heads over a graph network of the memory store, and evaluates it only over the legal joint actions so masking is exact. It warm-starts the policy by imitating a Belady oracle and assigns credit for delayed, query-time payoffs with a mix of oracle imitation and process supervision [11].

Having built the formulation, we take seriously the empirical question it invites—*is this decision actually learnable?*—and answer it on production KV traces rather than on synthetic workloads. The answer turned out to depend less on the model than on *when the model is asked*.

A natural way to evaluate this decision, and the one three revisions of this paper used, fixes a split instant: observe a prefix, build features from it, place every block, and bill the result against the future. Feature construction can only describe a block that has been seen — and on these traces 45–46% of blocks have no pre-split access at all, while the hindsight oracle they are scored against ranks them anyway. Those blocks carry **95% of the oracle's advantage**. A policy confined to the remaining 5% looks flat however well it ranks, which is precisely what our earlier "ranking does not convert into placement" result measured. We retract it.

Moving the decision to each block's *arrival*, placing it the moment it first appears from the session context available at that instant, changes what is within reach. Every arrival-time rule we test captures between **45%** and **85%** of the oracle's advantage on the production traces, where a policy placing only the blocks present at the split, even a clairvoyant one, cannot pass 5.3% and 4.1%. Within that range the order a rule ranks new blocks in hardly matters: arrival order, a learned ranker and a random order land within about a point of each other once their magnitudes are equal. What matters is the reuse a rule assumes for a block it has never seen, which moves the result by 30 to 39 points. The design implication is to build an **admission point**, decide at arrival, and treat that assumption as the first-order design choice.

Reaction is competitive for a related reason. Tiering rules that forecast nothing at all — LRU, LFU and a fill-once cache — reach 75–99% of Belady's hit rate on one Mooncake trace and 40–94% on the other, and the whole adaptive-replacement family (ARC, LeCaR, CACHEUS) beats the best of them by *at most 4.6 points*. All of them adapt on a ghost hit, and 76–78% of blocks here are never reused, so most evictions are correct and emit no signal: ARC gets one on 3.5–7.3% of misses. A population of singletons starves the channel adaptive replacement runs on.

We report how much measurement discipline this took, because it is the part most likely to transfer, and because it is not a flattering story. Three claims were retracted, the last two only under an adversarial review we ran with AI reviewer agents, *after* the error catalogue below had been written: a headline that randomized sampling showed had never been measurable, the decoupling result above, and an LRB column that turned out to measure random eviction because a training buffer was truncated to its tail. Later still, after every number check passed, an audit of our own arrival-time experiment found that its training labels, its split and its belief magnitudes all used information a deployed policy would not have; correcting them withdrew four readings this paper used to make, and a later re-verification found that its order comparison had been a magnitude comparison, withdrawing two more. In every case the aggregate looked reasonable, which is the point — none of our checks asked whether a reported quantity was arithmetically able to be what it claimed, and none asked whether a stored result was older than the program that wrote it.

a) Contributions.:

- **(C1)** A frame, not an evaluated system: agent-memory placement as one budget-constrained MDP over representation $\times$ tier $\times$ timing with a serve-time (L, C, B) knob (Section III), and a control-plane sketch for it (Section IV). The production traces exercise the tier axis only; we offer the formulation as the frame the measurements are read against, and do not claim to have evaluated it.
- **(C2)** A timing result on production traces. A split-instant evaluation protocol makes 95% of a hindsight oracle's advantage unreachable, because 45–46% of blocks do not exist yet when it decides (Section V-B); inside it, a better ranker helps monotonically but over a 5% sliver (Section V-D). Deciding at each block's *arrival* lets every rule we test capture 45–85% of the advantage, and separates the two parts of a belief: the order new blocks are ranked in moves the result by about a point, the magnitude assumed for them by 30–39 (Section V-C). On a synthetic control the margin over the split-time ceiling is not robust.
- **(C3)** Measurements of reactive tiering on both sources against the adaptive-replacement family (ARC, LeCaR, CACHEUS) and a reimplemented LRB — the best of them beating the best plain rule by at most 4.6 points of Belady across eight operating points — together with an eviction-signal inversion that makes a policy tuned on one source a poor default for the other, and a measured explanation: KV traces are dominated by one-time blocks, which starve the ghost-hit channel the three adaptive policies run on and leave most of LRB's training labels censored (Section V-F).
- **(C4)** A reusable evaluation protocol: twenty-five measurement defects that each manufactured a plausible result here, grouped into nine mechanisms, and the cheap controls that catch them — including the rule that a null is reportable only alongside the sample size that would have resolved it, and the rule that every edge of the source $\rightarrow$ artifact $\rightarrow$ figure $\rightarrow$ paper chain needs its own freshness check (Section V-G).

We position STOA against prior work along the three axes it unifies in Table I; no prior system occupies their intersection—deciding representation, tier, *and* timing, learned, under a

System	Repr.	Tier	Timing	Task util.	Budget cond.	Joint credit
MemOS [1]	○	○	○	○	○	–
Memory-R1 [3]	–	–	✓	✓	○	○
Mem-as-Act. [4]	–	–	✓	✓	○	✓†
LMCache [5]	–	✓	○	○	○	–
Cold-RL [8]	–	✓	–	○	○	✓†
Sleep-time [10]	–	–	✓	✓	○	–
STOA (proposed)	∗	∗	∗	∗	∗	∗

TABLE I

NO PRIOR SYSTEM DECIDES REPRESENTATION, TIER, *and* TIMING JOINTLY—LEARNED, UNDER A SHARED BUDGET, WITH TASK UTILITY AS THE OBJECTIVE. ✓: LEARNED; ○: FIXED/HEURISTIC; –: NOT APPLICABLE. †: CREDIT FOR ONE AXIS ONLY (CONTENT OR EVICTION). ∗: **PROPOSED IN SECTION III–IV AND *not evaluated in this paper*** — THE ROW STATES AN INTENDED DESIGN, NOT A MEASURED CAPABILITY, AND SECTION V GIVES REASONS TO DOUBT PARTS OF IT.

shared budget, with task utility as the objective.

b) Artifact.: Every number in this paper is regenerated on CPU, without a paid API, by the released package at https://github.com/suanlab/stoa; `make verify` re-derives each one from its stored result in a few minutes, and `make reproduce-full` regenerates those results from the public traces.

II. MOTIVATION: WHY NOW

a) The substrates exist; the orchestration does not.: By late 2025 every mechanism STOA needs had been built—in isolation. MemOS [1] exposes a MemCube abstraction with primitives to migrate an item between plaintext, activation, and parameter forms. LMCache [5] tiers KV-cache bytes across GPU, CPU, and remote storage. M+ [9] maintains a latent memory with a co-trained retriever, and sleep-time compute [10] shows that precomputing during idle windows cuts test-time cost by roughly $5\times$. RL memory managers [3], [4] learn what to write and edit. What is absent is the layer that *decides*, per item and under a shared budget, which of these mechanisms to invoke and when—a scheduler whose objective is downstream task utility rather than cache miss-rate.

b) A worked coupling example.: Consider a single fact accessed k times over a session (placeholder costs from our simulator). As plaintext on CPU it costs nothing to write but $\sim$800 tokens on every serve, so its serving cost grows as $800k$ tokens. Promoted once to a latent representation it costs a one-off consolidation but serves at $\approx$0 tokens thereafter. For a cold fact (k=1) the promotion never pays off; for a hot fact ($k \gg 1$) it is decisively cheaper—and the crossover depends on the tier (which sets read latency), on the timing (paying the promotion online vs. amortizing it in a sleep window), and on the residual token budget B. The three axes cannot be set independently, and the right setting is data-dependent—exactly the regime where a learned policy beats a fixed heuristic.

c) Why a control plane, and why learned.: Hand-tuned migration rules (as in today's memory-OS systems) cannot track this crossover across workloads, budgets, and backbones. But the decision is not intractable: it is a budget-constrained scheduling problem, and adjacent problems—cache replacement [6], [8] and cluster scheduling [7]—have repeatedly been shown learnable from traces. STOA applies that lesson to the joint representation–tier–timing decision, packaged as a control plane so it can sit above existing stores and tiers without replacing them.

III. PROBLEM FORMULATION

A. The per-item decision

An agent accumulates memory items $\mathcal{M} = \{m_1, \ldots, m_N\}$—facts, dialogue turns, tool results, summaries. For each item a memory system must answer three questions that are today decided separately: *representation* $\rho \in \{\text{plaintext}, \text{vector}, \text{graph}, \text{latent}, \text{parameter}\}$, the semantic form (trading fidelity, serving-token cost, write cost); *tier* $\tau \in \{\text{GPU}, \text{CPU}, \text{disk}, \text{remote}\}$, the physical location (trading read latency and capacity); and *timing* $t \in \{\text{online}, \text{sleep}, \text{noop}\}$, *when* the placement is (re)materialized.

The central claim is that these axes are **coupled**. Promoting an item to latent or parameter is an expensive offline write but then cheap and fast to serve; leaving it as plaintext on disk is cheap to write but slow to read and token-heavy. The optimal representation depends on the tier and on reuse; the optimal tier depends on the representation's size and serving cost. No axis can be optimized in isolation.

B. A budget-constrained MDP

We cast the orchestrator as a budget-constrained MDP solved by a learned policy π_θ.

a) State.: For each candidate item m_i the state $s_t^{(i)}$ concatenates: (a) *semantic* features—embedding, recency, access frequency, a provenance/version tag, a predicted next-access time (a Belady-style forecast); (b) *systems* features—current representation and tier, item size, and per-tier occupancy/pressure; (c) *budget* features—residual L, C, B; and (d) *query* context. Items are the nodes of a *memory graph* (edges: co-access, entity links) perceived jointly.

b) Action.: Each item is assigned one action $a = (\rho, \tau, t)$. The raw space has $|\rho|\cdot|\tau|\cdot|t| = 5\cdot4\cdot3 = 60$ combinations, but some are illegal and are **masked**: a parameter promotion is sleep-only (never inline online), and latent/KV memory is meaningful only on GPU/CPU. Masking removes $4+6 = 10$ combinations, leaving a fixed legal set of $|\mathcal{A}| = 50$ per item. Legality is defined once and is load-bearing: both the action enumeration and the policy's output distribution are defined over exactly these 50 actions.

c) Transition.: Executing a realizes a cost tuple $(\text{lat}, \text{cost}, \text{tok}, \text{fresh})$ from a transition model calibrated on tiering traces. Timing sets the economics: a sleep action amortizes the one-off promotion cost off the serving path; a noop serves the item from its current form at no write cost.

d) Objective.: We maximize expected task utility subject to serve-time budgets:

$$\max_{\theta} \mathbb{E}\Big[\sum_t U(a_t)\Big] \text{ s.t. Latency} \le L,\ \text{Cost} \le C,\ \text{Tokens} \le B, \tag{1}$$

where U is a downstream reward (QA accuracy, or a step-level process reward) realized only when the item is *used* to answer a query—many steps after placement. We relax the constraints with learned duals:

$$r_t = U - \lambda_L \,\text{lat} - \lambda_C \,\text{cost} - \lambda_B \,\text{tok} + \beta \,\text{fresh}. \tag{2}$$

The operator sets (L, C, B) at serving time and the policy adapts, so a *single control knob* moves the system along its accuracy–latency–cost frontier—the key operational difference from a statically configured memory system, and the property we probe as RQ3.

C. A factored policy over the memory graph

A joint head over 50 actions per item, across many items, is combinatorial. We factor the policy over a shared trunk, $\pi_\theta(a \mid s) \propto \pi_{\text{rep}}(\rho)\,\pi_{\text{tier}}(\tau)\,\pi_{\text{time}}(t)$, and evaluate it **only over the 50 legal joint actions**: the three factor logits are summed for each legal a and softmaxed over that set. This keeps the policy factored (three small heads) while making the induced distribution *masking-exact*—it never places probability on an illegal combination. The trunk is a graph network over the memory graph (Section IV), so co-accessed items inform one another's placement rather than being decided independently. Whether this policy class can actually be *fit* to useful accuracy is an empirical question, not a property of the formulation; Section V answers it.

IV. The STOA Control Plane

STOA is a *control plane*: it sits above a MemOS-style store, an LMCache-style tiering layer, and an M+-style latent path, and decides how each memory item is placed without replacing those substrates (Figure 1). A step of the loop takes the current set of candidate items and their systems telemetry, emits one action per item, and issues the corresponding store/tier calls; the offline branch additionally drains a consolidation queue during idle windows.

a) Perception.: A graph network encodes the memory store as a graph whose nodes are items and whose edges are co-access and entity links, together with per-tier occupancy telemetry, into a per-item embedding. Acting on the whole batch through a graph network—rather than scoring items independently—lets co-accessed items inform one another's placement and avoids per-item combinatorics, following Decima's use of a GNN for scheduling [7].

b) Policy.: A factored actor emits the three decisions (ρ, τ, t) per item through separate representation, tier, and timing heads, combined and softmaxed over exactly the 50 legal joint actions so masking is exact (Section III). A constrained critic—a value head—estimates the value of a placement under the current budget (L, C, B) and serves as the baseline for policy-gradient updates. A separate admission/consolidation head selects which items enter the sleep-time queue for offline promotion.

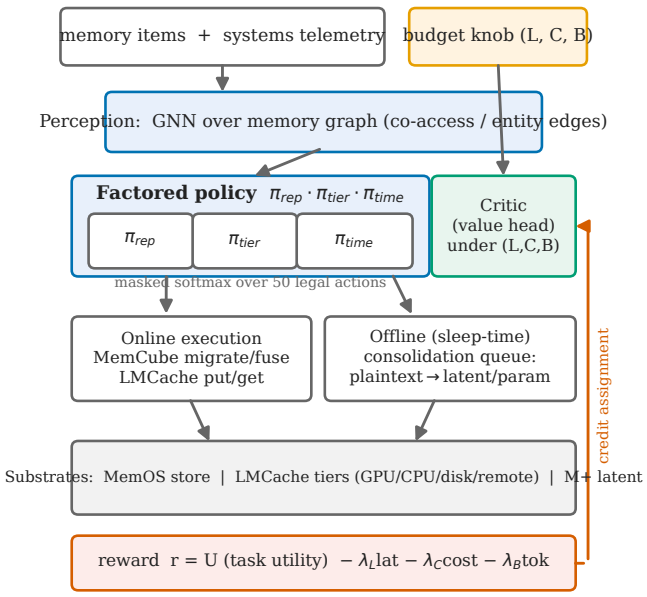


Fig. 1. The STOA control plane. A GNN perceives the memory graph; a factored, masking-exact policy and a budget-conditioned critic emit one action per item; online and sleep-time branches execute against the underlying store, tiers, and latent path. Task-utility reward flows back through credit assignment.

c) Execution.: Online actions are executed inline on the serving path as MemCube migrate/fuse operations and LMCache put/get calls. Offline (sleep-time) actions run during idle windows: promoting plaintext to latent or parameter forms, or summarizing a cluster of turns into a graph, amortized off the serving path [10], [12].

d) Credit assignment.: The hard part is that a placement's payoff arrives many steps later—at query time—and memory edits fracture the trajectory prefix, so naive policy gradients are high-variance and biased. STOA combines three ingredients: (a) **Belady-oracle imitation** [6] to warm-start the policy from hindsight-optimal placements computed offline from full future access traces; (b) **process supervision** with a learned critic that assigns dense, step-level rewards to migration and consolidation actions [11]; and (c) **segment-level advantages** over the trajectory segments induced by memory edits [4]. Training proceeds offline-RL first—safe on logged traces, in the style of production cache-eviction RL [8]—and then guarded online. **None of this machinery is evaluated in this paper.** Section V measures the decision the architecture would make — whether a better belief about which items will be reused buys a better placement — and finds that it does not. We therefore report the formulation and the measurement, and leave the policy itself as a design that our own evidence does not currently motivate building.

V. What Placement Has To Get Right

Section III frames placement as a learnable budget-constrained MDP. This section reports what we found when

we tried to learn it on production KV traces, and it is organized around a defect rather than a result, because the defect came first and everything else follows from it: the split-instant protocol, which we used for three revisions, makes 95% of the achievable benefit unreachable, so a policy measured inside it looks flat however well it ranks (Section V-B). Moving the decision to each block's arrival puts most of the benefit within reach, and there the belief assigned to an unseen block matters far more than how blocks are ranked (Section V-C). We then report what a better ranker buys once the metric is repaired (Section V-D), where reaction stands (Section V-F), and, at length because it is the most transferable part, the protocol, including the three claims we retracted along the way (Section V-G).

A. Sources and references

We use **Mooncake** [13], replayed production traces from a general LLM serving stack (conversation, 12,031 requests; toolagent, 23,608), and **kv-cache-tester**, anonymized agentic coding conversations (200 of 739 fetched). Each request lists the hash ids of the KV blocks it touches, so a repeated hash is a genuine reuse. Both sources record wall-clock request timestamps. Block ids in kv-cache-tester are conversation-scoped, so we namespace them before pooling; not doing so invents cross-conversation reuse. Table II summarizes both sources.

Two references bound every policy, both billed on the true future: a **hindsight oracle** that ranks and places knowing future access counts, and a **frequency heuristic** that assumes the future looks like the observed prefix. "Headroom" is $(\text{cost}_{\text{heur}} - \text{cost}_{\text{oracle}})/\text{cost}_{\text{heur}}$; "captured" is the fraction of that interval a learned policy recovers. Cost combines latency, money and context tokens with weights $\lambda_L = 0.1$ per ms, $\lambda_C = 1$ per USD and $\lambda_B = 0.0005$ per token, which the artifact documents as illustrative unit-normalising values and has not changed since its first commit.

B. The protocol decides what is reachable

A natural way to evaluate placement is to fix a *split instant*: observe the trace up to some point, build features from that prefix, place every block, and bill the result against what the future turns out to contain. It is what we ran for three revisions of this paper, and it has a structural defect that no aggregate computed inside it can reveal.

The defect is general, not particular to our setup. Whenever a workload admits new items after the split (that is, whenever it is not a closed working set), a split-time policy has no opinion about those items while the hindsight oracle it is scored against ranks them freely. The reported "fraction of headroom captured" then divides by a quantity that is partly unreachable, and the unreachable share grows with the arrival rate. Any study reporting that fraction without bounding it is reporting a number whose denominator it has not characterized.

Feature construction can only describe a block that has been seen. On these traces **45.2% and 46.1% of blocks have no pre-split access at all**, and a policy therefore has no opinion about them — but the hindsight oracle it is scored against ranks them anyway, using knowledge of a future in which they exist. Those blocks carry **94.7% and 95.9%** of the oracle's total advantage. The denominator of every "captured headroom" number we reported was, in other words, almost entirely unreachable by construction:

	conversation	toolagent
blocks with no pre-split access	45.2%	46.1%
full hindsight gap	93,383	94,714
attainable gap (visible blocks only)	**4,996**	**3,900**
share no causal policy can reach	**94.7%**	**95.9%**

It is not an artifact of where we cut. Sweeping the split instant from 0.3 to 0.7 of the trace leaves the unreachable share at **91.8–98.4%** on conversation and **93.0–98.8%** on toolagent — and it is *worse* at earlier splits, where more of the trace is still to come. Nor is it an artifact of the cost weights: scaling λ_L or λ_B by 0.1 or by 10 leaves it at 91.8–99.2%. The pathology is a property of the protocol, not of our choices within it.

It is a property of arrivals, not of production. Mooncake's third released workload is synthetic (public datasets, Poisson arrivals), and we use it as a control rather than pool it. Only 24.6% of its blocks are new at the midpoint, against 45–46% on the production traces, and the unreachable share falls with them: **65.0%** at the midpoint and **45.3–83.5%** across splits. Within every one of the three traces the share rises monotonically as the split moves earlier; across traces the level differs, production's new blocks carrying more of the oracle's advantage than synthetic ones at a similar new-block share. Even where only 8% of blocks are new, almost half the advantage is out of reach.

A policy confined to 5% of the prize will look flat no matter how well it ranks, and that is what our earlier "decoupling" result measured. We retract it (Section V-G).

a) The diagnostic is one line.: Compute the oracle's advantage twice, once with clairvoyance over every item and once restricted to the items the policy could have had an opinion about, and take the ratio. If it is far from one, the protocol is the finding. It costs a single extra placement pass, it needs no new data, and running it on the first day would have saved this paper three revisions. We ship it as `attainable_ceiling` in the artifact and recommend it as a standard reporting requirement for placement and admission studies: *state the reachable share alongside any captured fraction*.

C. Deciding on arrival, not at a split

About 95% of the oracle's advantage lives in blocks that do not exist yet when a split-time policy decides. The alternative is an earlier decision: place each block *when it first appears*, from context available at that instant (the shared-prefix session's age, request rate and recency, the block's position within its request, the request's length). Nothing the policy uses reads the future. The arrival population is exactly the blocks first

seen at or after the billing split. The model is fitted on blocks first seen before it, each labelled by its reuse within a fixed window that closes before the split. A belief has an order and a magnitude, and the arms vary them separately. The three *order* arms put every new block's score onto one magnitude multiset, the previous period's analogue (blocks first seen before the split, counted up to it), so they differ only in which block gets which value. The two *magnitude* arms give every new block one constant, the mean or the median of that multiset, and admit them in arrival order. Blocks that already exist keep the frequency heuristic's belief in every arm.

	split	order (one multiset)			magnitude	
trace	ceiling	arrival	learned (AUC)	shuffled	mean	median
conversation	5.3	59.0	59.5 (0.56)	58.4	75.0	45.5
toolagent	4.1	58.7	58.7 (0.61)	57.9	84.7	45.6
synthetic	35.0	44.1	50.5 (0.75)	42.7	52.6	33.6

Percent of the full hindsight gap. The split-time ceiling is clairvoyant on every block visible at the split; every arrival-time arm is causal.

Deciding at arrival puts most of the advantage within reach. On the production traces every arrival-time arm captures 45–85% of the oracle's advantage, where a policy that places only the blocks present at the split, even a clairvoyant one, cannot exceed 5.3% and 4.1%. On the synthetic control, where only a quarter of blocks are new at the split, the margin narrows and is not robust: the arms span 33.6–52.6% against a 35.0% ceiling.

What an arrival rule assumes matters more than how it ranks. With magnitudes held fixed, arrival order, a learned ranker and a random order land within 1.1 and 0.9 points of each other on the production traces; only on the synthetic control does the ranker lead, by 6.4 points. Changing the constant every new block receives from the median to the mean (1.0 to 1.54 and 2.14) moves the result by 29.5 and 39.1 points: at 1.0 a new block ties with, and is placed behind, every pre-split block seen once. An admission point therefore has to decide how far to trust a block it has never seen against one seen once, the same lesson as Section V-D: the magnitude convention moves the result more than ranking does. Arrival order is itself weakly predictive (AUC 0.55–0.58), because in a finite trace blocks arriving in the first quarter see 2.3–4.6 times the future accesses of those in the last, so even the small differences in order are confounded with exposure (Section VI).

Two earlier versions of this experiment were wrong, and six of their readings are withdrawn. The first leaked: its training labels counted accesses after the split (34.3% and 46.3% of training-label mass), its train/eval cut was a different instant from its billing split, its belief magnitudes were the evaluation blocks' true future counts, and it gave pre-split blocks a belief of zero in the arrival arms. The second compared a first-come-first-served arm holding every new block at the mean against learned and shuffled arms on the spread multiset, so its "order" comparison was mostly a magnitude comparison. Withdrawn: that a learned ranker captures less than FCFS, that FCFS captures 87% and 50% of what is reachable, that arrival order is worth 14 points, that timing recovers nothing on the synthetic control, that FCFS leads the ranker by 2 to 26 points, and that this lead reflects exposure time (Section V-G).

The design implication we can support is coarse: build an **admission point** rather than a predictor, and treat the belief it assigns an unseen block as the first-order design choice. Baleen [14] is the one system among those we survey that decides admission rather than eviction, and it optimizes a downstream system cost rather than miss rate.

D. What a better ranker buys inside the split protocol

Within the split-instant protocol, does a stronger model convert into a better placement? Each rung is trained on one trace and evaluated on the other, so the AUCs are cross-trace. With ties broken by tier speed and every belief quantile-matched onto one magnitude multiset, the scored blocks' own pre-split counts, so that only the ordering varies:

rung	AUC (conv / tool)	% of attainable (conv)	(tool)
constant	0.50 / 0.50	−48.6	−60.6
random	0.49 / 0.50	−62.7	−284.0
log(count)	0.64 / 0.71	0.0	0.0
logistic	0.81 / 0.93	+50.7	+55.0
GBDT	0.80 / 0.91	+47.6	+44.9
MLP	0.82 / 0.93	+52.6	+59.5
ceiling	1.00	100	100

Ranking quality converts monotonically on both traces. log(count) reproduces the frequency heuristic exactly, since it ranks by the same counts the magnitudes come from, and each better model recovers more, the best about half of what is attainable. But what is attainable is the 5% of Section V-B: the strongest model gains 2.8% and 2.5% of the oracle's advantage. A previous version of this table placed every rung on the evaluation blocks' true future counts, which no deployed policy has; there every rung fell below the heuristic and conversion failed on one trace.

a) The magnitude convention moves the result more than ranking does.: Running each rung twice, once on the scale its own scores naturally produce and once on the common multiset, *with the ordering identical in both*, gives:

trace	rung	native	common	Δ
conversation	logistic	+74.1	+50.7	23.4
	log(count)	−40.7	0.0	−40.7
	random	−84.5	−62.7	−21.8
toolagent	logistic	+87.8	+55.0	32.7
	log(count)	−25.2	0.0	−25.2
	random	−665.4	−284.0	−381.5

Percent of the attainable ceiling. The two columns differ only in the magnitudes attached to an identical ranking.

The same policy, with the same ordering, scores +87.8% or +55.0% depending on a choice of scale, and across rungs the scale moves the result by **22 to 381 points**, more than any

	Mooncake (serving)	kv-cache-tester (agentic)
requests analysed	12,031 / 23,608	4,215 (40 conv.)
blocks touched exactly once	75.8 / 78.5%	15%
mean reuse per block	1.6 / 2.2	26
blocks per request	24 / 17	∼1,900
block size (tokens)	512	64
headroom, heuristic vs. oracle	80%	68–73%

TABLE II
THE TWO SOURCES, AT THE SCALES ACTUALLY ANALYSED. MOONCAKE PAIRS ARE CONVERSATION / TOOLAGENT; ITS HEADROOM AVERAGES THE TWO (79.9% AND 79.8%). THE AGENTIC COLUMN IS THE FIRST 40 CONVERSATION FILES, EXCEPT HEADROOM, WHICH IS THE MEAN OVER FOUR RANDOMIZED DRAWS OF 20 AND OF 40 CONVERSATIONS (SECTION V-E). ITS DESCRIPTIVE STATISTICS ARE SAMPLE-DEPENDENT: THE ONE-TIME FRACTION RANGES 7.5–21% OVER 20–200 CONVERSATIONS. AN EARLIER REVISION TOOK THIS COLUMN'S HEADROOM FROM NESTED-PREFIX SAMPLES (81–94%), THE PROTOCOL SECTION V-E RETRACTS.

	sorted-prefix samples (as first reported)	4 random draws mean [observed range]
20 conversations	—	-541% $[-1207, +78]$
40 conversations	$+88\%$	-67% $[-234, +66]$
70 conversations	$+87\%$	—
100 conversations	-9%	—
150 conversations	-48%	—

TABLE III
HEADROOM CAPTURED BY A LEARNED PLACEMENT POLICY ON AGENTIC TRACES, THE SAME PIPELINE MEASURED TWO WAYS. LEFT: SORTED-PREFIX SAMPLES, WHICH ARE NESTED AND DRAWN FROM A CORPUS WHOSE PER-CONVERSATION STATISTICS VARY THREEFOLD ALONG THE FILE INDEX — THE NUMBERS WE FIRST REPORTED, AND THEIR REVERSAL. RIGHT: RANDOMIZED DRAWS AT THE SAME SIZES, SUMMARIZED BY THEIR OBSERVED RANGE — FOUR DRAWS CANNOT SUPPORT A PARAMETRIC INTERVAL. THE SPREAD IS AN ORDER OF MAGNITUDE WIDER THAN ANY EFFECT, SO THIS QUANTITY WAS NEVER MEASURABLE UNDER THIS PROTOCOL. WE REPORT IT AS A CAUTIONARY MEASUREMENT.

ranking difference above. We report the common-scale column, where only ordering varies, and make the methodological point: **"fraction of headroom captured" is not a quantity until the magnitude convention is stated**, and none of our earlier drafts stated it.

E. A measurement that was never a measurement

We also attempted the obvious follow-up (fit a richer per-block model on the agentic source and place with it) and first obtained what looked like a strong positive: $+66$ to $+88\%$ of the headroom captured, reproducing across what we took to be two samples. It was not a result, and the way it failed is worth more than the number would have been.

The two samples were *sorted prefixes* of the conversation corpus, so the smaller was contained in the larger; and file index correlates strongly with conversation shape (mean requests per conversation falls from 140 to 37 between files 1–20 and 71–150). Extending the same prefix to 100 and 150 conversations drove the captured headroom to -9% and -48% — a sign reversal. Re-drawing the samples *at random* rather than as prefixes then showed why: four random draws at 40 conversations land between -234 and $+66$ points, and at 20 conversations between -1207 and $+78$ (Table III). The spread dwarfs the effect by an order of magnitude. We quote the observed range rather than a confidence interval on purpose: with four draws a parametric interval rests on a normality assumption these data cannot support, and *captured* is a ratio whose denominator varies per draw. An earlier revision of this paper reported ± 126 here, computed with the normal quantile against a standard deviation estimated from four points; the correct $t(3)$ half-width is ± 205. That error was ours. Under a correct sampling protocol there was never a measurement here to report.

We keep this in the paper for two reasons. It is the control that our own protocol (Section V-G) lacked, and running it costs minutes; and it bounds what the rest of our numbers are worth, since every other result in this section is also a single draw of a parameter we chose.

F. Where reaction stands

The alternative to forecasting is to react. On the full Mooncake traces, reactive tiering reaches 75–99% of Belady's hit rate on the toolagent trace and 40–94% on open-ended conversation using LRU, LFU or a fill-once cache alone; admitting the adaptive and learned policies below widens the per-operating-point envelope only to 76–99% and 42–94% (Figure 2).

The interesting question is not the envelope — that is an oracle over policies, not something anyone deploys — but whether any *single* policy attains it, and whether the adaptive designs built for exactly this situation beat plain rules. We therefore run the family: **ARC** [15], which shifts a recency/frequency split on ghost hits; **LeCaR** [16] and **CACHEUS** [17], which replace ARC's arithmetic update with regret-based multiplicative weights over two experts; and the learned **LRB**, which is reported separately below because its numbers carry an implementation band the others do not.

trace	tier	best pure	ARC	LeCaR	CACHEUS
conversation	2%	39.8	29.4	**42.3**	35.9
	5%	53.3	**57.9**	56.8	53.5
	10%	76.0	**77.5**	77.2	75.3
	20%	**94.1**	87.4	93.9	88.8
toolagent	2%	74.9	70.7	**75.8**	73.9
	5%	81.3	**83.6**	83.0	82.9
	10%	91.9	92.3	**92.9**	91.8
	20%	99.3	96.3	**99.3**	97.6

Percent of Belady's hit rate. "Best pure" is the better of LRU, LFU and a fill-once cache at that operating point.

Across eight operating points and four designs, **the best adaptive or learned policy beats the best pure rule by at most 4.6 points of Belady**, and at two points by nothing at all. Which design wins also moves: LeCaR at the tightest tier on both traces and ARC at 5%; above that, ARC and then plain LRU on conversation, LeCaR on toolagent. That is the substantive finding — not that these policies fail, but that on KV-block workloads the whole adaptive-replacement family

collapses into a band a few points wide, well short of the oracle at the small tiers where the decision actually matters.

The diagnostics say why, and the reason is specific to KV workloads. Every policy in this family adapts on a *ghost hit* (a miss on a block it recently evicted), whether it updates a split (ARC) or an expert weight (LeCaR, CACHEUS). But 76–78% of blocks in these traces are never reused at all, so most evictions are correct and produce no signal. Measured, ARC receives an adaptation signal on **3.5–7.3% of misses**; the regret-based policies read the same channel, which is why they land in the same band rather than above it. What signal it does get is not merely sparse but, at the smallest tier, directionally wrong: at a 2% tier ARC drives p to 90% recency on both traces, while the rule that actually wins there is frequency. A workload whose blocks are mostly singletons starves the very mechanism adaptive replacement runs on.

a) And the learned cache policy loses to both — but only as a range.: The strongest counter-argument to this paper is LRB [18], which learns a relaxed-Belady objective from a far richer feature set than our four trace-local statistics and won on production CDN traces. We reimplement it: GBDT regression on log time-to-next-request over inter-access deltas and exponentially decayed counters, sampled eviction, online retraining from a sliding memory window, right-censored labels.

Our first version of it measured something else entirely. The training buffer was capped by truncating to its tail; labelled rows arrive one at a time as blocks are re-accessed while censored rows arrive in one contiguous burst per checkpoint, so from the third retrain onward every training label was identical, the regressor returned a constant, and eviction degenerated to a uniform draw over the sampled candidates. We report the repaired numbers and, because two defensible repairs disagree, we report them as a band:

trace	tier	LRB (repaired)	ARC	below ARC?
conversation	2%	27.6–29.2	29.4	yes
	5%	30.9–32.2	57.9	yes
	10%	45.0–45.9	77.5	yes
	20%	75.8–76.5	87.4	yes
toolagent	2%	68.8–69.0	70.7	yes
	5%	71.1–71.4	83.6	yes
	10%	80.9–83.0	92.3	yes
	20%	94.6–94.9	96.3	yes

Percent of Belady. The band spans two repairs of the truncation bug: subsampling all accumulated rows, and dropping rows older than the memory window first. We adopt the second, which is what a "sliding memory window" means.

What the band supports. The two repairs agree to within 2.1 points at every operating point, and **LRB is below ARC at all eight operating points under both.** That ordering is the claim we make; the numbers are context for it. Both repairs and the retracted behaviour are selectable in the artifact, so the band is regenerated rather than reported. We do not claim the band bounds every reasonable reimplementation: a third repair produced in an AI-agent review we ran (see the Acknowledgment) reported a higher figure, and since its code is not ours to ship, we neither reproduce nor rely on it.

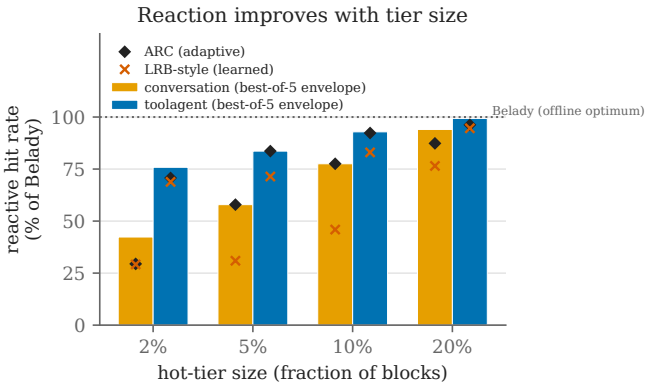


Fig. 2. Reactive tiering as a fraction of Belady's hit rate, full Mooncake traces. Bars are a per-operating-point best-of-five envelope, an oracle over policies rather than a policy. Diamonds are ARC, which meets the envelope at 5% and 10% but falls short at both extremes; crosses are the LRB-style learned policy, which is below ARC everywhere.

The mechanism appears to be the one that limits ARC, seen from the other side. LRB learns from labels the trace supplies when a sampled block is requested again; a block that never returns yields a *right-censored* row, which says only "not within the window" and nothing about when. At the default window **83–88% of LRB's training rows are censored**, and 88–91% at a shorter one. A population dominated by singletons starves the ghost-hit channel that adaptive replacement runs on and the regression target that learned replacement runs on, for the same underlying reason.

Three caveats bound this. It is our reimplementation of the published mechanism, not the authors' artifact. KV blocks are uniform-size, so byte miss ratio collapses to object miss ratio and LRB loses one of its structural advantages over LRU. And the hyperparameter sweep we ran to check that the result was not a tuning artifact was run on the *broken* implementation, so it tested the robustness of random eviction; we have not repeated it.

The eviction signal also inverts by source. On the agent traces LRU reaches 0.83–0.96 of Belady at a 5% tier while LFU stays near 0.29–0.31; on Mooncake the ordering reverses below the working-set knee. An eviction policy tuned on one source is a poor default for the other — and, as both the ARC and the LRB result show, reaching for a more sophisticated policy does not by itself resolve the choice.

G. What the measurement required

Every positive result we obtained before the final protocol was an artifact. We report the mechanisms because they generalize, and because the later ones were found only after the earlier catalogue had been written — two by an adversarial review we ran with AI reviewer agents (see the Acknowledgment), and the rest by re-verification passes run over a paper that had already passed every check it defined for itself.

a) Reference-frame mismatches (six).: Two quantities that must share a frame (the same split instant, ordering, tie rule, or question set) were silently defined on different ones: a demand signal computed from the questions being scored; policy features derived from whole-trace access counts; an oracle that inherited the heuristic's ordering and so showed zero headroom; a train/test split taken over access-event indices rather than time, which on real traces falls past the end of the trace; a ranking metric that broke ties by label, inflating a raw count feature from AUC 0.51 to 0.94; and features built at one split instant while costs were billed at another.

b) Retracted claims (three).: Each survived our own checks and was found later — the last two by the AI-agent review, after the catalogue below had already been written. **(i)** A headline "+66–88% of headroom captured on agentic traces" that reversed sign at a larger sample and that randomized draws showed had never been measurable (Section V-E). **(ii)** A *decoupling* claim (prediction improves while placement does not) that was an artifact of normalizing against an oracle ranking blocks a causal policy cannot see (Section V-B); normalized against what is attainable, ranking quality converts monotonically on both traces, though only over the sliver Section V-B leaves reachable. **(iii)** A reported LRB column that measured *random eviction*: the training buffer was truncated to its tail, censored rows arrive in one contiguous burst per checkpoint, and from the third retrain onward every training label was identical, so the regressor returned a constant and eviction degenerated to a uniform draw. Two defensible repairs of that bug disagree by up to 2.1 points (Section V-F). We therefore report the column as a band and state the conclusion (LRB below ARC at all eight operating points) only at the coarseness that band supports. An earlier draft of this paragraph said the repairs disagree by "19 points"; they disagree by 2.1, and the 19 came from a comparison this artifact cannot reproduce.

A later re-verification pass "falsified" two sub-claims of the arrival result using numbers that, an audit then showed, came from a leaky version of the experiment itself; under the corrected design both questions turn out to be entangled with exposure time and are left open (Section V-C).

The common shape is worth naming: in all three the *aggregate looked reasonable*. A flat captured fraction, a plausible interval, a learned policy landing below ARC — each is what the hypothesis predicted, and none of our checks compared the quantity against what it was arithmetically able to be.

c) Free-parameter extrapolations (three).: Each was a curve read from one point, in a parameter fixed for convenience. A reactive claim taken at a single cache size reversed under a sweep. A headroom taken at 1,500 requests moved by full length from 72.2% to 79.8% on toolagent and from 70.4% to 79.9% on conversation, and the placement benefit read off it peaked mid-sweep rather than at the longest trace. And a learned-placement result taken at 40 and 70 conversations, *nested prefixes* of a corpus whose per-conversation statistics vary threefold along the file index, reversed sign at 150, from $+88\%$ to -48%, and took a headline claim with it.

d) Believing a report over an observation (one).: An edit script targeted a heading an earlier revision had removed, matched nothing, and printed success anyway.

e) Derived and parallel surfaces outliving their source (three).: A result is not one file but a chain: source $\rightarrow$ artifact $\rightarrow$ figure $\rightarrow$ paper. Fixing a link does not refresh what follows it, and nothing in our pipeline compared adjacent links. Three artifacts the paper cited were generated *before* a fix to the code that produced them (two of them denominators), which is how the two falsified sub-claims above reached a draft. After adding a freshness check on that first edge and observing it pass, we found the plotted figures were fifteen days older than the artifacts they draw: the text carried the corrected ladder while the figure still showed the retracted one, because the paper embeds the PDF and a number-checker cannot see inside it. A freshness check on one edge reads like a freshness check on the chain; it is not, and a guard that has only ever passed has not been tested. The same claim also lives on surfaces that are not downstream of the paper but parallel to it (a repository README, a lab notebook), and correcting the document that *makes* a claim does not correct the documents that *recorded* it. Our notebook stated a falsified conclusion in the present tense a hundred and fifty lines above its own retraction of it. Which repair applies depends on what the surface is: an assertion is updated, a record is banner-marked and left intact.

f) Summaries of checked tables, unchecked (one).: Our checker re-derives every number in this paper from its artifact. It verified all eight rows of the LRB band above and passed, while a sentence elsewhere in the paper described that same band as "two repairs disagreeing by 19 points" — the repairs disagree by 2.1, and the 19 belonged to a different comparison altogether. Every element of the table was right and the summary of it was false. Row-level verification certifies a table, not the prose that aggregates it; widths, spans, maxima and "at most" claims need their own assertions, derived from the same artifact. The deeper form of the same gap: every check we had asked whether the *correct* value was present, none whether a *retracted* one was absent. Putting two withdrawn figures back into our abstract left all of them passing, because the live values were still in place further down; an absence check now fails it.

g) Unverified interventions read as evidence (one).: We concluded something from an intervention that never took effect, in three forms: a banner deleted only in part, a paragraph edited around a wrong number, a mutation whose target string was absent and which therefore reported "no test caught this" — indistinguishable from a real coverage gap. A negative result from an intervention you did not confirm landed is not a result. Our harnesses now assert the change applied before reading the outcome, and the guards are themselves tested in both directions: four had lived in a script where a refactor could have deleted any one silently. A test's *name* is a claim about coverage that nothing checks — ours named for the three-routine tie invariant compared two of them — and the only way to learn what a test pins is to break the thing and watch it fail — having first checked that the break changed

the computation and not merely the file.

h) Emphasis without verification (one).: Our checker re-derives every number it is told about, and for the life of this paper it was not told about the one the abstract leads with. The "fraction of the oracle's advantage that is unreachable", quoted in the abstract, in C2, in Section V-B and here, sat in no artifact and in no assertion, and when finally recomputed it was built from two different references: its attainable gap measured against the frequency heuristic, its full gap against something 1.72× larger on both traces. The qualitative finding survived; the figure moved from 97% to 95%. The guard is a coverage test rather than a correctness one: extract every emphasised number in the source and fail if nothing asserts it. It answers "what is nothing watching?", which is a different question from "is what we watch correct?" and the one we had never asked.

i) Leaks and confounds that no number check can see (six).: Found after every number check passed, five by an audit and the sixth by the re-verification of its fixes. The arrival-time experiment labelled its training blocks with accesses after the split; cut its training and evaluation sets at a different instant from the one it billed at; set every policy's belief magnitudes from the evaluation blocks' true future counts, a leak of the label distribution that four other experiments shared; and gave pre-split blocks a belief of zero in the arrival arms while the baseline placed them by frequency. Its "ceiling" then mixed past-window and future-window counts in one greedy ordering, and a causal rule exceeded it. And its corrected version's "order-only" controls differed from the first-come-first-served arm in belief magnitude as well: tied scores receive their tie block's mean, so a constant score is a magnitude choice. Each produced a self-consistent number that matched its artifact exactly, so none was visible to a check that compares the paper to its artifacts.

j) The controls.: Assert that the oracle beats the heuristic it is compared against; keep a no-learning baseline of the same functional form as the learned one; check that no single feature outperforms the fitted model; before concluding that a signal is unlearnable, vary model capacity while holding the protocol fixed, so "the model was too weak" is excluded rather than assumed (Section V-D); compare reference frames explicitly rather than trusting that two code paths agree; verify an edit landed rather than that it reported success; sweep every parameter chosen for convenience before stating a result that depends on it — the one we lacked; and, before reporting any null, compute the sample size that would have resolved it, because "we could not tell" and "there is nothing there" are different claims and only the first is ours to make. The harness ships with all of these wired in, and a checker that re-derives every number in this paper from its artifact and fails if the two have drifted — which caught four rounding drifts and one claim that had quietly mixed two hyperparameter settings.

H. Limitations

No real vLLM+LMCache deployment was measured, so any cost-expressed quantity should be read as a range. We tightened what that range means. Our first sensitivity analysis scaled every tier latency by a common factor, which moves the absolute scale and leaves the *ratios* between tiers untouched — and the ratios are what a placement policy responds to, so it never tested the parameter that mattered. We therefore derive the table instead: KV bytes per block follow exactly from a model's attention shape ($2 \times$ layers $\times$ kv_heads $\times$ head_dim $\times$ dtype per token), and per-tier latency is that divided by a declared device-class bandwidth plus a fixed overhead. Sweeping three attention shapes against five device classes puts headroom at **60–83%**, which contains the 80% our hand-written table gives — the conclusion survives, but two things about the table do not. Its GPU:CPU latency ratio is 1:10 where every device class but the deliberately flat one gives 1:80 to 1:250, an order of magnitude in the ratio the policy is most sensitive to; and **6 of the 15 configurations reorder the hierarchy** so that latency order and capacity order disagree, which our fixed per-tier capacity table cannot express at all. Calibrating against a live deployment remains the open gate; this is the honest intermediate, not a substitute. The *timing* axis of Section III (when an item is materialized) is not exercised at all; the title's timing is the instant a placement is decided, a different quantity, and the *representation* axis is not exercised here either: a pilot over three textual fidelities is in the artifact, but it depends on a paid hosted model, and this paper reports only what the artifact regenerates without one. Both sources are KV-block hash streams from transformer prefix caching, so they are independent in provenance but not in mechanism; with one exemplar per putative regime we cannot separate workload character from vendor, date, or block size (512 vs. 64 tokens). Sampling is not randomized, every placement result is a single deterministic run, and no confidence interval is reported for them, though trace length, split instant and cost weights are now swept rather than fixed. The arrival-time comparison is a static placement over a finite window, in which arrival order is entangled with exposure time, so it ranks protocols but not admission rules. Of the learned and adaptive cache literature we evaluate ARC [15], LeCaR [16], CACHEUS [17] and a reimplementation of LRB [18]; Hawkeye, Mockingjay and Baleen are positioned in Section VII but not run, the first two because a KV block hash carries no analogue of the program counter their predictors are indexed by. Our LRB is the published mechanism rebuilt from the paper, not the authors' artifact (as are our LeCaR and CACHEUS, whose CR-LFU expert we approximate by LFU), and the uniform block size of both sources removes one of LRB's advantages over LRU, so the underperformance we report should be read as a statement about this workload rather than about these designs.

VI. Open Problems

Our result closes one question and sharpens several others.

a) What should an admission point believe about a block it has never seen?: Section V-C shows that deciding at arrival puts most of the oracle's advantage within reach, and that the result then turns on one number: the reuse assumed for

a new block relative to a block seen once. We set it by hand, to the mean or the median of the previous period; learning it per session or per request, causally, is the obvious next model, and the one that would turn the 45–85% range into a point. Ranking new blocks against each other is a second-order question, and comparing rules on it needs an evaluation that gives every block the same exposure, a rolling window or a fixed post-arrival horizon, because in a finite trace early arrivals have more time to be reused. Whatever such an evaluation leaves unclaimed has to come from something the per-item formulation does not express: admission under contention, coordinated eviction across concurrently-live sessions, or migration timed against idle windows.

b) Calibration, and what a mis-specified cost model hides.: Our tables are placeholders, and we saw concretely how misleading that is: under them the per-item optimal action is identical for every reused block, which collapses a three-axis decision into a one-dimensional sort and makes every policy look alike. Fitting the tables to a real vLLM+LMCache deployment — the only GPU-gated step in this work — is a precondition for any claim about the representation and timing axes, which our traces do not exercise at all.

c) Aggregate working set, not per-request footprint.: Reactive sufficiency is governed by how much of the *concurrently live* working set a tier holds, not by one request's block count: holding tier size fixed in absolute blocks and raising the number of pooled conversations still degrades the ratio. Characterizing that surface — tier size against concurrency, across sources — would give an operator a sizing rule, which is the most directly actionable thing this line of work could produce.

d) Composing placement with retrieval.: Placement sets latency and cost; retrieval sets which facts reach the model. The open problem is the composition — a retriever running over a tiered store whose hot set a scheduler maintains, where a retrieval miss on a cold tier costs latency rather than accuracy. Neither literature evaluates that joint object, and the KV traces here cannot: they record block reuse, not which facts a query needed.

e) Sampling as a first-class experimental variable.: Three of the defects in Section V-G were extrapolations from a single value of a parameter fixed for convenience, and the most damaging reversed a headline claim's sign. For trace-driven work we would now require of ourselves, and suggest for the area: randomized rather than prefix sampling, several disjoint draws with intervals, and a reported sweep over every quantity the design fixes — trace length, pool size, tier size, split fraction — before any claim that depends on one of them.

f) Safety and governance.: A memory written by tools and other agents is an attack surface, and provenance must survive consolidation so a fact can be traced, invalidated, or unlearned. A budget-constrained scheduler is a natural enforcement point; doing that without giving back the frontier is unexplored.

VII. Related Work

We organize prior work by the three axes STOA unifies; Table I summarizes which axis each line decides and by what means.

a) Representation and memory-OS.: MemOS [1] and MemGPT [2] treat memory as an operating-system problem, exposing abstractions (MemCube, virtual context) and primitives to migrate items between plaintext, activation, and parameter forms; M+ [9] adds a co-trained latent memory. These provide the *mechanism* to change representation and tier but decide *when* by heuristic; STOA supplies the learned scheduler they lack.

b) Content and RL-based management.: Memory-R1 [3] and Memory-as-Action [4] learn, with RL, *what* to add, update, or delete and how to edit context, and the latter handles the trajectory-fracturing that edits cause. But they operate over a single fixed representation and never choose a physical tier or schedule offline consolidation—orthogonal to, and composable with, STOA.

c) Systems tiering and ML-for-systems.: Serving systems already make prefix KV blocks reusable: PagedAttention [19] manages them in pages that requests can share, SGLang [20] reuses them across requests through a prefix tree, and CachedAttention [21] and Mooncake [13] keep them across a hierarchy of storage tiers, with hand-designed policies for what stays where; our traces come from the last of these. LMCache [5] tiers KV-cache bytes to cut miss-rate; H2O [22] evicts by attention score. The learnability of such decisions is established: PARROT [6] imitates a Belady oracle for cache replacement, Cold-RL [8] does eviction with offline RL in production, and Decima [7] schedules clusters with a GNN policy. STOA inherits these techniques but changes the objective from miss-rate to downstream *task utility*, and the action from a single axis to the joint representation $\times$ tier $\times$ timing decision.

d) Offline consolidation.: Sleep-time compute [10] and LightMem [12] show the value of precomputing and consolidating memory offline, but treat it as a global mode rather than a per-item, budget-aware timing decision—the third axis STOA learns.

e) Learned and adaptive caching.: Two lines bear directly on our results and we position against them explicitly, because our findings reproduce rather than extend them. **LRB** [18] learns a relaxed-Belady objective for CDN caching from a feature vector built out of the deltas between an object's past request times — the same family as the session recency and rate features our arrival-time ranker uses. Timing as a cache-scheduler input is therefore established practice; what we add is evidence about *when* a placement decision is taken, not a new signal to take it with. **ARC** [15] adaptively balances recency against frequency precisely because neither dominates across workloads; the recency-versus-frequency inversion we measure between our two sources (Section V-F) is that observation restated on KV blocks, and an adaptive policy of the ARC family is the baseline a deployment should reach for rather than either pure rule.

The rest of the learned-cache literature divides by what it learns *from* and what it decides, and the division explains why our result is not a contradiction of it. LeCaR [16] and CACHEUS [17] learn no per-item model at all: they run experts (LRU/LFU, and in CACHEUS scan- and churn-resistant variants) and update expert weights from regret observed on a ghost list. That is the same evidence channel ARC uses, and Section V-F measures how thin it is here — 3.5–7.3% of misses carry any signal — so the family inherits the limitation rather than escaping it. We ran both to check, rather than leaving it as a prediction, and both land in the same few-point band as ARC. LeCaR is nonetheless the best single policy at the tightest tier on both traces, which is the one place the regret formulation visibly pays. Hawkeye [23] and Mockingjay [24] are the closest antecedents to what we attempted: both supervise a predictor with Belady replayed over history, Hawkeye as a binary cache-friendly/averse label and Mockingjay as a multiclass reuse-distance estimate. They operate at CPU last-level-cache granularity, where a program counter identifies a reuse pattern; a KV block hash carries no analogous instruction identity, which is the structural reason their signal does not simply port. Baleen [14] is the one that decides admission and prefetching rather than eviction, and the one that optimizes a downstream system cost (disk-head time) instead of miss rate — the closest precedent for the multi-objective reward of Section III, and the direction we think is right even though we do not evaluate it here.

Of these we evaluate LRB, LeCaR and CACHEUS (Section V-F) — and the prediction above is what we measured: both expert policies land in the same few-point band as ARC rather than above it. Hawkeye, Mockingjay and Baleen are positioned but not run.

In short, each line optimizes one axis and freezes the others, and none decides representation, tier, *and* timing jointly under a shared budget (Table I). Our contribution is not a better policy but a measurement: on production KV traces the usual split-instant protocol leaves most of a hindsight oracle's advantage out of any policy's reach, deciding at each block's arrival brings much of it back into reach, and we report the protocol needed to see that.

VIII. Conclusion

For every item an agent remembers, three coupled questions (in what representation, on which tier, and when to materialize it) are today answered separately by three research communities. We offered a single budget-constrained formulation as a frame and measured one of its axes, placement across tiers, on production KV traces.

The finding is about when to decide, and it is first a finding about evaluation. Evaluated at a split instant, as is natural, more than nine-tenths of a hindsight oracle's advantage is out of reach of any policy that places the blocks present at the split, for every split we tried and under tenfold changes to the cost weights; inside that protocol a better ranker helps monotonically, but over a sliver. Deciding at each block's arrival puts most of the advantage within reach: every arrival-time rule we test captures 45–85% of it on both production traces, and the reuse a rule assumes for a block it has never seen matters far more than the order it ranks such blocks in. Reactive tiering, forecasting nothing, already delivers much of the achievable hit rate, and the adaptive-replacement family adds little to it, because the signal it adapts on is scarce when most blocks are never reused.

We arrived here only after removing twenty-five measurement defects, each of which had produced a plausible result, including leaks and an unfair control in our own arrival-time experiment that were found after every number check had passed. We report their mechanisms and the controls that catch them, because the same traps are open to anyone measuring a memory scheduler. The next steps are an admission rule that learns the belief it gives unseen blocks, an evaluation with equal exposure per block, a cost model calibrated against real tiers, and the habit of asking what a reported fraction's denominator could have been.

Acknowledgment

Use of AI-generated content. An AI assistant (Anthropic's Claude, Opus 5 and Opus 5.5 models, used through the Claude Code agentic environment) was used extensively throughout this work. It wrote most of the code in the released artifact, designed and ran most of the experiments and analyses, drafted and revised most of the text, and generated the figures from the stored results; its contributions are recorded commit by commit in the artifact's history. The "adversarial review" and the audit referred to in Section V-G were internal exercises in which AI agents were prompted to act as reviewers; neither was peer review, and one figure the review produced (a third LRB repair) is neither reproduced nor relied on here. Every number in this paper is re-derived from the released artifact by a checker shipped with it. The initial concept and research idea were proposed by the author. The author takes full responsibility for the content of this paper.

References

[1] Z. Li *et al.*, "MemOS: A memory OS for ai system," *arXiv*, 2025.
[2] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv*, 2023.
[3] S. Yan *et al.*, "Memory-R1: Enhancing LLM agents to manage and utilize memories via reinforcement learning," *arXiv*, 2025.
[4] Y. Zhang *et al.*, "Memory as action: Autonomous context curation for long-horizon agentic tasks," *arXiv*, 2025.
[5] Y. Cheng, Y. Liu *et al.*, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," *arXiv*, 2025.
[6] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, "An imitation learning approach for cache replacement," *ICML*, 2020.
[7] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," *SIGCOMM*, 2019.
[8] A. Gupta and A. Bhayani, "Cold-RL: Learning cache eviction with offline reinforcement learning for NGINX," *arXiv*, 2025.
[9] Y. Wang, D. Krotov, Y. Hu *et al.*, "M+: Extending MemoryLLM with scalable long-term memory," *ICML*, 2025.
[10] K. Lin, C. Snell, Y. Wang, C. Packer, S. Wooders, I. Stoica, and J. E. Gonzalez, "Sleep-time compute: Beyond inference scaling at test-time," *arXiv*, 2025.
[11] G. Xiong, Q. Jin, X. Wang *et al.*, "RAG-Gym: Optimizing reasoning and search agents with process supervision," *arXiv:2502.13957v1*, 2025.

[12] J. Fang *et al.*, "LightMem: Lightweight and efficient memory-augmented generation," *arXiv (ICLR 2026)*, 2025.

[13] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a kvcache-centric architecture for serving llm chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, traces: github.com/kvcache-ai/Mooncake, FAST25-release. [Online]. Available: https://www.usenix.org/conference/fast25/presentation/qin

[14] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, and G. R. Ganger, "Baleen: ML admission and prefetching for flash caches," in *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. Santa Clara, CA: USENIX Association, Feb. 2024, pp. 347–371. [Online]. Available: https://www.usenix.org/conference/fast24/presentation/wong

[15] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *2nd USENIX Conference on File and Storage Technologies (FAST)*, 2003. [Online]. Available: https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache

[16] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan, "Driving cache replacement with ML-based LeCaR," in *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. Boston, MA: USENIX Association, Jul. 2018. [Online]. Available: https://www.usenix.org/conference/hotstorage18/presentation/vietri

[17] L. V. Rodriguez, F. B. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan, "Learning cache replacement with CACHEUS," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, Feb. 2021, pp. 341–354. [Online]. Available: https://www.usenix.org/conference/fast21/presentation/rodriguez

[18] Z. Song, D. S. Berger, K. Li, and W. Lloyd, "Learning relaxed belady for content distribution network caching," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 529–544. [Online]. Available: https://www.usenix.org/conference/nsdi20/presentation/song

[19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[20] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[21] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic *et al.*, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *USENIX Annual Technical Conference (ATC)*, 2024.

[22] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," *NeurIPS*, 2023.

[23] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2016, pp. 78–89.

[24] I. Shah, A. Jain, and C. Lin, "Effective mimicry of Belady's MIN policy," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 558–572.